Feed-Forward Neural Network Using Keras and TensorFlow

Dataset: MNIST handwritten digits
Optimizer: Stochastic Gradient Descent (SGD)

Objectives

- Load and preprocess the MNIST training and testing datasets.
- Define a feed-forward neural network using Keras.
- Train the model using SGD.
- Evaluate the model and plot training loss and accuracy.

> Run the cells from top to bottom. The first run may download MNIST from the internet.

a. Import the necessary packages

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.21.0
```

b. Load the training and testing data

```
# Load MNIST: 60,000 training images and 10,000 test images
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)

# Normalize pixel values from [0, 255] to [0, 1]
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Convert digit labels (0–9) to one-hot encoded vectors
num_classes = 10
y_train_cat = to_categorical(y_train, num_classes)
y_test_cat = to_categorical(y_test, num_classes)

print("Preprocessing completed.")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)
Preprocessing completed.
```

c. Define the network architecture using Keras

```
model = Sequential([
    Input(shape=(28, 28)),
    Flatten(),                        # Convert 28 x 28 image into 784 values
    Dense(128, activation="relu"),    # Hidden layer
    Dense(64, activation="relu"),     # Hidden layer
    Dense(10, activation="softmax")   # Output layer: 10 digit classes
])

model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

```
Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)
```

d. Compile and train the model using SGD

```
# SGD = Stochastic Gradient Descent
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    x_train,
    y_train_cat,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.6656 - loss: 1.3102 - val_accuracy: 0.8775 - val_loss: 0.5898
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8674 - loss: 0.5285 - val_accuracy: 0.9053 - val_loss: 0.3684
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8890 - loss: 0.4089 - val_accuracy: 0.9162 - val_loss: 0.3086
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8998 - loss: 0.3611 - val_accuracy: 0.9242 - val_loss: 0.2786
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9069 - loss: 0.3319 - val_accuracy: 0.9270 - val_loss: 0.2606
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9124 - loss: 0.3111 - val_accuracy: 0.9300 - val_loss: 0.2465
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9166 - loss: 0.2946 - val_accuracy: 0.9340 - val_loss: 0.2348
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9206 - loss: 0.2807 - val_accuracy: 0.9373 - val_loss: 0.2253
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9244 - loss: 0.2687 - val_accuracy: 0.9402 - val_loss: 0.2175
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9269 - loss: 0.2581 - val_accuracy: 0.9422 - val_loss: 0.2085
```

e. Evaluate the network

```
test_loss, test_accuracy = model.evaluate(x_test, y_test_cat, verbose=0)

print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")
```

```
Test loss: 0.2437
Test accuracy: 92.99%
```

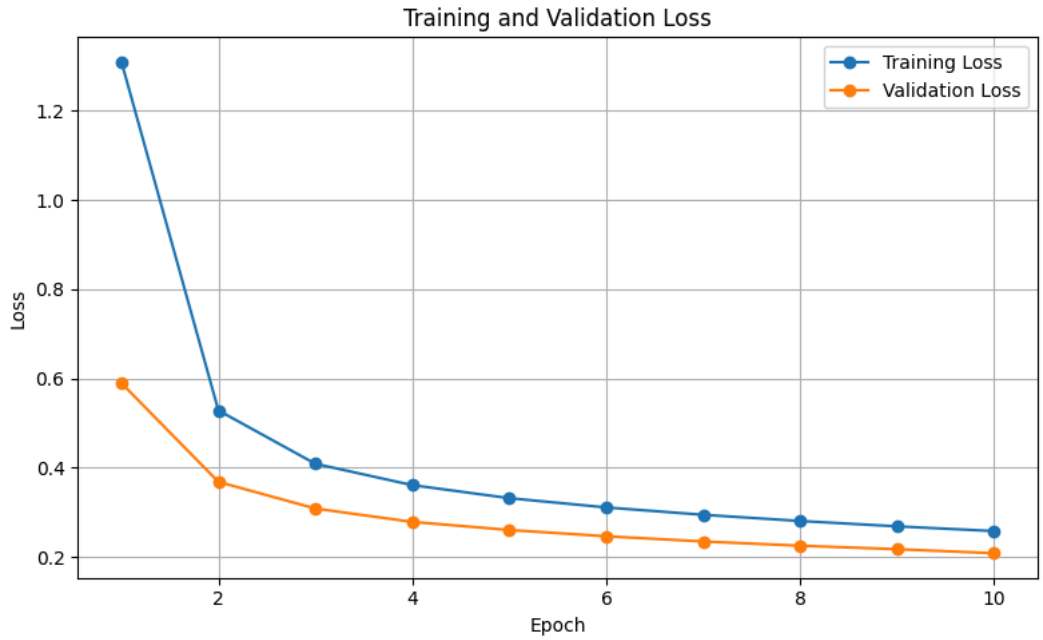
f. Plot the training loss and accuracy

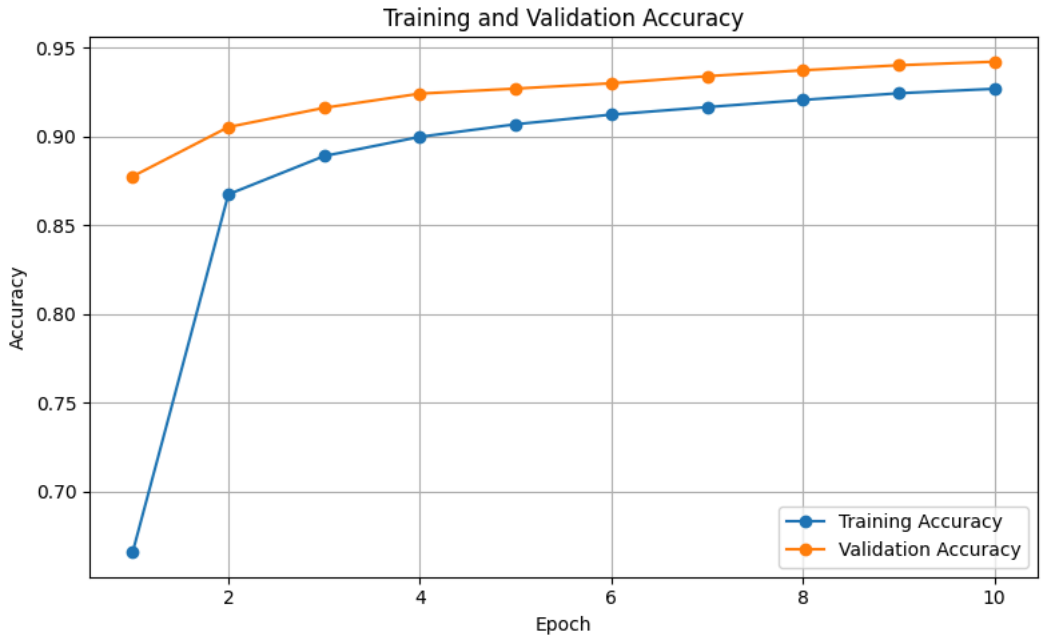
```
epochs = range(1, len(history.history["loss"]) + 1)

# Plot training and validation loss
plt.figure(figsize=(8, 5))
plt.plot(epochs, history.history["loss"], marker="o", label="Training Loss")
plt.plot(epochs, history.history["val_loss"], marker="o", label="Validation Loss")
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
```

```
plt.show()

# Plot training and validation accuracy
plt.figure(figsize=(8, 5))
plt.plot(epochs, history.history["accuracy"], marker="o", label="Training Accuracy")
plt.plot(epochs, history.history["val_accuracy"], marker="o", label="Validation Accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```





Conclusion

A feed-forward neural network was implemented using TensorFlow and Keras on the MNIST dataset. The model was trained with SGD, evaluated on the test dataset, and its training/validation loss and accuracy were plotted.

Note: The exact test accuracy may vary depending on the TensorFlow version and training environment.